

Especificación de Requisitos de Software (SRS)

Sistema: SGED — Sistema de Gestión para la Escuela Deportiva ProFútbol **Versión del documento:** 1.2 (Entrega Final, etiqueta `v1.0.0` — revisado por última vez tras la reestructuración de paquetes académico / deportivo / seguridad del 2026-07-29) **Estructura:** basada en ISO/IEC/IEEE 29148:2018

Repositorio: https://github.com/darcalleg/SGED_APPWEB

Nota de redacción (resuelve OBS-01, Entrega 1A). El docente observó que los requisitos funcionales estaban redactados como títulos ("Registro de estudiantes") en vez de como requisitos. En este documento **todo requisito funcional se enuncia con la forma "El sistema deberá..."**, con un identificador único, una prioridad y un criterio de verificación comprobable.

1. Introducción

1.1 Propósito

Este documento especifica los requisitos funcionales y no funcionales de SGED, una aplicación web para la gestión administrativa y deportiva de la escuela de fútbol formativo ProFútbol. Está dirigido al equipo de desarrollo y al docente evaluador del Proyecto Fin de Curso.

1.2 Alcance

SGED cubre cuatro dominios:

- Seguridad y acceso** — personas, usuarios, roles y autenticación.
- Gestión académica/administrativa** — registro y mantenimiento de estudiantes y sus categorías.
- Dominio deportivo** — entrenadores, horarios, sesiones de entrenamiento, asistencia y evaluación diaria del desempeño.
- Inventario** — catálogo de artículos deportivos (uniformes, balones, implementos), control de stock por movimientos y asignación de artículos a estudiantes o entrenadores (RF-27 a RF-30, ver ADR-003).

1.3 Estado de implementación (declaración de honestidad)

Esta sección resuelve OBS-12 (Entrega 1B), donde el docente observó que el informe describía funcionalidad que no existía en el repositorio. Para evitar repetir ese error, cada requisito indica explícitamente su estado real, verificable en el código:

Estado	Significado
 Implementado	Existe endpoint REST funcional, con pruebas y evidencia de ejecución.
 Modelado	El esquema de base de datos existe y está migrado (Flyway), pero aún no se expone vía API REST.
 Planificado	Solo especificado en este documento; sin esquema ni código.

Ningún requisito marcado  o  debe interpretarse como funcionalidad entregada.

1.4 Definiciones y acrónimos

Término	Definición
---------	------------

Categoría	Grupo etario de competencia, definido por un rango de edad (p. ej. "Sub-12"). Desde la reestructuración de paquetes es una entidad propia (<code>deportivo.categorias</code>) con <code>edad_min/edad_max</code> , no un texto libre.
Baja lógica	Marcar un registro como inactivo (<code>activo = FALSE</code>) sin borrarlo físicamente.
JWT	JSON Web Token (RFC 7519), credencial de sesión firmada.
JTI	Identificador único de un JWT, usado para revocarlo.
RFID	Identificación por radiofrecuencia; medio previsto para marcar asistencia.
ProblemDetail	Formato de respuesta de error de RFC 7807 / RFC 9457.
Representante	Padre, madre o tutor legal de un estudiante menor de edad.

1.5 Referencias

- ISO/IEC/IEEE 29148:2018 — Requirements engineering.
- ISO/IEC 25010:2011 — Modelo de calidad de producto software.
- RFC 9110 — HTTP Semantics.
- RFC 7519 — JSON Web Token.
- RFC 9457 — Problem Details for HTTP APIs.
- OWASP Top 10:2021.

2. Descripción general

2.1 Perspectiva del producto

SGED es un sistema cliente-servidor de tres capas:

- **Frontend:** Angular (SPA), servido por nginx con terminación TLS en `:8443`.
- **Backend:** API REST en Spring Boot 3.2 (Java 21), puerto `:8080`.
- **Persistencia:** PostgreSQL 16 (esquemas `seguridad`, `academico` y `deportivo`) y Redis 7 (caché y lista de revocación de tokens).

Orquestación reproducible vía Docker Compose con imágenes fijadas por digest SHA-256.

2.2 Actores del sistema

Actor	Descripción	Rol técnico
Administrador	Gestiona usuarios, estudiantes y configuración. Único actor con permisos de escritura sobre estudiantes.	ADMINISTRADOR
Entrenador	Consulta estudiantes de sus categorías, registra asistencia y evaluación diaria.	ENTRENADOR
Recepcionista	Registra estudiantes, cobra membresías/pagos diarios y emite el QR de asistencia.	RECEPCIONISTA
Representante	Tutor legal del estudiante; consulta informes de sus representados.	REPRESENTANTE
Estudiante	Marca su propia asistencia escaneando el QR.	ESTUDIANTE

No existe un rol genérico de "usuario estándar": cada cuenta se crea con uno de estos roles reales (`rol` es obligatorio en `POST /api/auth/registro`). Los cinco están sembrados en `db/seed.sql` y son los que evalúan las anotaciones `@PreAuthorize` del código.

2.3 Restricciones de diseño

- **RD-01.** El sistema deberá ejecutarse íntegramente mediante contenedores Docker, sin instalación manual de dependencias en la máquina anfitriona.
- **RD-02.** Las operaciones elementales (CRUD simple, consultas paginadas) deberán resolverse con Spring Data JPA; las operaciones de agregación y actualización masiva con criterio de negocio deberán ejecutarse en el motor de base de datos mediante procedimientos almacenados versionados.
- **RD-03.** El sistema no deberá construir sentencias SQL por concatenación dinámica de cadenas en ninguna capa.

3. Requisitos funcionales

3.1 Módulo de seguridad y acceso

RF-01 — Registro de usuarios *El sistema deberá permitir que un usuario con rol ADMINISTRADOR registre nuevas cuentas de usuario, asociándolas a una persona y a uno o más roles.*

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
- **Origen:** `POST /api/auth/registro` — `AuthController.java:63`
- **Restricción de acceso:** `@PreAuthorize("hasRole('ADMINISTRADOR')")`
- **Verificación:** un usuario no autenticado o sin rol ADMINISTRADOR deberá recibir `401 / 403` . Prueba: `AuthServiceTest.registroExitoso` , `AuthServiceTest.registroEmailDuplicado` . Evidencia OWASP A01: `docs/mediciones/sec/a01-acceso-roto.txt` .

RF-02 — Autenticación de usuarios *El sistema deberá autenticar a un usuario mediante nombre de usuario y contraseña, y deberá emitir la credencial de sesión exclusivamente en una cookie `HttpOnly` , `Secure` y `SameSite=Strict` , sin exponer el token en el cuerpo de la respuesta ni en almacenamiento accesible por JavaScript.*

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
- **Origen:** `POST /api/auth/login` — `AuthController.java:99`
- **Verificación:** la respuesta deberá contener `Set-Cookie` con los tres atributos y no deberá contener el JWT en el cuerpo. Pruebas: `AuthServiceTest.loginConCredencialesCorrectas` , `AuthServiceTest.loginConContraseñaIncorrecta` .

RF-03 — Cierre de sesión con revocación efectiva *El sistema deberá permitir cerrar la sesión, y deberá invalidar el token emitido registrando su identificador (JTI) en una lista de revocación con tiempo de vida igual al tiempo restante del token, de modo que un token robado antes del cierre de sesión no siga siendo aceptado.*

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
- **Origen:** `POST /api/auth/logout` — `AuthController.java:143` ; `RedisBlacklistService.java`
- **Verificación:** pruebas `RedisBlacklistServiceTest.revocar_guarda_el_jti_con_el_ttl_restante` , `RedisBlacklistServiceTest.estaRevocado_true_si_existe_la_clave` .
- **Nota:** resuelve OBS-07 y OBS-09 (Entrega 1B), donde se observó que la lista de revocación existía pero no estaba cableada a ningún endpoint.

RF-04 — Renovación de sesión *El sistema deberá permitir renovar una sesión vigente mediante un token de refresco, sin exigir que el usuario vuelva a introducir sus credenciales.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
- **Origen:** POST /api/auth/refresh — AuthController.java:164
- **Verificación:** prueba JwtServiceTest.refresh_token_valido .

RF-05 — Consulta de la sesión activa El sistema deberá permitir que el cliente consulte los datos de la sesión en curso (nombre de usuario, nombre completo y rol) a partir de la cookie de sesión, y deberá responder 401 cuando no exista sesión válida.

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Origen:** GET /api/auth/me — AuthController.java:181
- **Verificación:** con sesión válida deberá responder 200 con {username, nombre, rol} ; sin sesión, 401 .

RF-06 — Limitación de intentos de autenticación El sistema deberá bloquear temporalmente los intentos de autenticación de un mismo usuario tras 5 fallos consecutivos dentro de una ventana de 15 minutos, y el contador no deberá reiniciarse con cada nuevo fallo dentro de esa ventana.

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Origen:** LoginAttemptService.java ; parámetros LOGIN_MAX_INTENTOS=5 , LOGIN_VENTANA_MINUTOS=15 (application.yml)
- **Verificación:** el sexto intento deberá responder 429 con cuerpo ProblemDetail . Pruebas: LoginAttemptServiceTest.bloqueada_al_alcanzar_el_limite , LoginAttemptServiceTest.fallo_subsiguiente_no_reinicia_ttl . Evidencia OWASP A07: docs/mediciones/sec/a07-rate-limit.txt .

RF-07 — Verificación de disponibilidad del servicio El sistema deberá exponer un endpoint público de comprobación de disponibilidad que no requiera autenticación.

- **Prioridad:** Baja · **Estado:** ☒ Implementado · **MoSCoW:** Could
- **Origen:** GET /api/auth/ping — AuthController.java:202
- **Verificación:** prueba AuthServiceTest.pingRespondePong .

3.2 Módulo de gestión de estudiantes

RF-08 — Listado paginado de estudiantes El sistema deberá permitir consultar el listado de estudiantes de forma paginada, indicando en la respuesta el número de página, el tamaño, el total de elementos y el total de páginas.

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Origen:** GET /api/estudiantes — academico/estudiante/controller/EstudianteController.java
- **Acceso:** ADMINISTRADOR , ENTRENADOR , RECEPCIONISTA
- **Verificación:** pruebas EstudianteControllerTest.listar_devuelve_pagina , EstudianteServiceTest.listar_devuelve_pagina_envuelta .

RF-09 — Consulta de estudiante por identificador El sistema deberá permitir consultar un estudiante por su identificador, y deberá responder 404 con cuerpo ProblemDetail cuando el identificador no corresponda a ningún registro.

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Origen:** GET /api/estudiantes/{id} — academico/estudiante/controller/EstudianteController.java

- **Verificación:** pruebas `EstudianteControllerTest.buscarPorId_existente` ,
`EstudianteControllerTest.buscarPorId_inexistente_da_404` ,
`EstudianteServiceTest.buscar_inexistente_lanza_404` .

RF-10 — Registro de estudiante *El sistema deberá permitir que un usuario con rol ADMINISTRADOR registre un nuevo estudiante asociado a una persona, una categoría y un estado general existentes, con un código de estudiante único y fecha de ingreso, creando de forma transaccional el registro correspondiente.*

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
- **Origen:** `POST /api/estudiantes` —
`academico/estudiante/controller/EstudianteController.java`
- **Acceso:** `@PreAuthorize("hasAnyRole('ADMINISTRADOR', 'RECEPCIONISTA')")` (esta nota decía solo ADMINISTRADOR; corregido 2026-08-12 para reflejar el código real — RECEPCIONISTA siempre pudo registrar estudiantes).
- **Verificación:** deberá responder `201` con el recurso creado. Pruebas:
`EstudianteControllerTest.crear_devuelve_201` ,
`EstudianteServiceTest.crear_persiste_persona_y_estudiante` .
- **Cambio respecto a la v1.0 de este documento:** el estudiante ya no se crea con nombre/apellido propios (esos viven en `Persona` , referenciada por `idPersona`); `EstudianteRequest` exige `idPersona` , `idCategoria` , `idEstadoGeneral` , `codigoEstudiante` y `fechaIngreso` , y admite opcionalmente `peso` y `altura` (ver hallazgo H-06 en `ETHICS.md`).
- **Frontend (2026-08-12):** se sirve desde la pantalla unificada `/personas` (`frontend/src/app/features/personas/`), que reemplaza a las antiguas `/estudiantes/registrar` y `/admin/crear-usuario` — ver `docs/superpowers/specs/2026-08-12-personas-unificado-design.md` .

RF-11 — Validación de la categoría del estudiante *El sistema deberá exigir que todo estudiante esté asociado a una categoría existente en el catálogo, mediante una clave foránea válida, y deberá responder `422 Unprocessable Entity` si la categoría indicada no existe o si falta.*

- **Prioridad:** Alta · **Estado:**  Implementado — **contenido reescrito el 2026-07-30** · **MoSCoW:** Must
- **Origen:** `EstudianteRequest.idCategoria` (`@NotNull`); `deportivo.categorias` como catálogo referenciado.
- **Verificación:** prueba `EstudianteControllerTest.crear_con_categoria_invalida_da_422` (pendiente de re-ejecutar contra el nuevo DTO — ver nota de cobertura en RNF-09).

Por qué cambió. La versión anterior de este requisito describía una validación de patrón de texto (`SUB-NN`) sobre un campo `VARCHAR` . Ese campo ya no existe: la categoría es ahora una entidad normalizada (`deportivo.categorias` , con `edad_min` / `edad_max`) referenciada por `idCategoria` . Se corrige el requisito para no describir una validación que el código ya no hace.

RF-11b — Registro de peso y altura del estudiante  Implementado sin resolución ética *El sistema permite registrar opcionalmente el peso y la altura de un estudiante al crearlo o actualizarlo, validando que sean valores positivos con hasta 3 dígitos enteros y 2 decimales.*

- **Prioridad:** No priorizado formalmente — apareció en la reestructuración de paquetes, no en un requisito previamente especificado. · **MoSCoW:** Won't (esta entrega) — pausado por el hallazgo H-06, no por olvido.
- **Origen:** `EstudianteRequest.peso` , `.altura` (`@DecimalMin` , `@Digits`); columnas `academico.estudiantes.peso/altura` .
- **Alerta:** este requisito se documenta pero **no se recomienda mantenerlo habilitado** sin resolver antes el hallazgo H-06 de `docs/etica/ETHICS.md` (dato de salud de un menor, sin finalidad ni base legal

documentada). Ningún caso de uso ni historia de usuario de este documento describía esta funcionalidad antes de que apareciera en el código.

RF-12 — Actualización de estudiante *El sistema deberá permitir que un usuario con rol ADMINISTRADOR actualice los datos propios de un estudiante existente (categoría, estado, código, fecha de ingreso, peso y altura).*

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Origen:** PUT /api/estudiantes/{id} —
academico/estudiante/controller/EstudianteController.java
- **Verificación:** prueba EstudianteControllerTest.editar_actualiza_estudiante .

RF-13 — Baja lógica de estudiante *El sistema deberá dar de baja a un estudiante marcándolo como inactivo, y no deberá eliminar físicamente el registro, con el fin de preservar el historial deportivo asociado.*

- **Prioridad:** Alta · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Origen:** DELETE /api/estudiantes/{id} —
academico/estudiante/controller/EstudianteController.java
- **Verificación:** deberá responder 204 y el registro deberá permanecer en la tabla con activo = FALSE .
Pruebas: EstudianteControllerTest.eliminar_devuelve_204 ,
EstudianteServiceTest.eliminar_hace_baja_logica .

RF-14 — Conteo de estudiantes activos por categoría *El sistema deberá informar el número de estudiantes activos de una categoría, identificada por su clave, y dicho conteo deberá calcularse en el motor de base de datos mediante un procedimiento almacenado versionado, no en la capa de aplicación.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
- **Origen:** GET /api/estudiantes/conteo/categoria/{idCategoria} —
academico/estudiante/controller/EstudianteController.java ;
academico.sp_contar_estudiantes_activos(p_categoria INT)
- **Acceso:** ADMINISTRADOR , ENTRENADOR
- **Verificación:** pruebas EstudianteControllerTest.contarActivos_delega_en_service ,
EstudianteServiceTest.conteo_por_categoria_delega_en_funcion_sql .
- **Justificación:** cumple RD-02 (agregación obligatoriamente en el motor).
- **Cambio respecto a la v1.0:** la ruta y el parámetro cambiaron de /conteo/{categoria} (texto) a /conteo/categoria/{idCategoria} (entero); el procedimiento se movió del esquema seguridad a academico y su parámetro de VARCHAR a INT .

RF-15 — Desactivación masiva por categoría *El sistema deberá permitir dar de baja lógica, en una sola operación transaccional, a todos los estudiantes activos de una categoría, e informar el número de registros afectados; dicha operación deberá ejecutarse mediante un procedimiento almacenado versionado.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
- **Origen:** POST /api/estudiantes/operaciones/desactivar-categoria —
academico/estudiante/controller/EstudianteController.java ;
academico.sp_desactivar_estudiantes_categoria(p_categoria INT)
- **Acceso:** @PreAuthorize("hasRole('ADMINISTRADOR')")
- **Verificación:** prueba EstudianteControllerTest.desactivarCategoria_delega_en_service .

3.2b Módulo de catálogos y cuentas (nuevo en esta revisión)

Recursos con CRUD propio que aparecieron con la reestructuración de paquetes y no tenían requisito documentado hasta ahora.

RF-23 — Gestión del catálogo de categorías *El sistema deberá permitir crear, listar, consultar, actualizar y eliminar categorías deportivas, cada una definida por un nombre y un rango de edad (mínima y máxima).*

- **Prioridad:** Alta (bloquea RF-10/RF-11) · **Estado:** ☒ Implementado · **MoSCoW:** Must
- **Origen:** `CategoriaController` (`/api/categorias` , 6 endpoints) — `deportivo/categoria/controller/CategoriaController.java`
- **Verificación:** `CategoriaServiceTest` (9 pruebas: paginación, alta, edición, baja lógica, validación de rango de edad), `CategoriaControllerTest` (7 pruebas: 200/201/204/400/404/422).

RF-24 — Gestión de cuentas de usuario como recurso propio *El sistema deberá permitir administrar cuentas de usuario (más allá del alta hecha en `POST /api/auth/registro`) de forma independiente.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
- **Origen:** `UsuarioController` (`/api/usuarios` , 5 endpoints) — `seguridad/usuario/controller/UsuarioController.java`
- **Verificación:** `UsuarioServiceTest` (paginación, username duplicado, alta con contraseña codificada, alta con rol asignado, rol inexistente da error, persona inexistente, baja lógica, edición de rol/usuario/contraseña, coherencia rol↔ficha), `UsuarioControllerTest` (200/201/204/400/422).
- **Cambio 2026-08-12:** `UsuarioRequest` agrega un campo `rol` opcional — si viene, se valida contra `seguridad.roles` y se asigna al crear (mismo criterio que `AuthController.registro` , pero sin forzar una `Persona` nueva). Permite darle acceso con cualquier rol a una `Persona` ya existente, no solo a las recién registradas. Frontend: pantalla unificada `/personas` (ver RF-10 y `docs/superpowers/specs/2026-08-12-personas-unificado-design.md`).
- **Cambio 2026-08-12 (edición y coherencia).** `PUT /api/usuarios/{id}` ahora sí aplica el cambio de `rol` (antes lo ignoraba) y la contraseña pasa a ser opcional: en blanco significa "no cambiarla". Además se valida la **coherencia rol↔ficha**: si la `Persona` tiene una ficha activa de `Estudiante/Entrenador/Representante`, su cuenta solo admite el rol correspondiente; sin fichas activas admite cualquiera (lo que permite crear la cuenta `ENTRENADOR` antes de la ficha). La guarda simétrica vive en `EstudianteService.crear` . Ver `docs/superpowers/specs/2026-08-12-validaciones-rol-usuario-design.md` y `2026-08-12-coherencia-rol-y-vinculo-representante-design.md` .

RF-25 — Gestión de personas *El sistema deberá permitir administrar los datos personales base (nombre, cédula, correo, teléfono, fecha de nacimiento) independientemente del rol que la persona tenga en el sistema.*

- **Prioridad:** Media · **Estado:** ☒ Implementado · **MoSCoW:** Should
- **Origen:** `PersonaController` (`/api/personas` , 6 endpoints) — `seguridad/persona/controller/PersonaController.java`
- **Verificación:** `PersonaServiceTest` (8 pruebas: paginación, búsqueda por cédula, unicidad de cédula/correo al crear y al editar, baja lógica), `PersonaControllerTest` (6 pruebas: 200/201/204/400/422).
- **Frontend (2026-08-12):** `Persona` es la raíz de la que cuelgan `Usuario` / `Estudiante` / `Entrenador` / `Representante` ; la pantalla `/personas` refleja esa jerarquía en vez de crear la `Persona` por separado en cada flujo. `RepresentanteController` también se abrió a `RECEPCIONISTA` para crear/vincular representantes (editar/eliminar sigue exclusivo de `ADMINISTRADOR`) — ver spec de diseño.

RF-26 — Consulta de estados generales *El sistema deberá exponer el catálogo de estados generales utilizables por usuarios y estudiantes.*

- **Prioridad:** Baja · **Estado:**  Implementado (solo lectura — 1 endpoint) · **MoSCoW:** Could
 - **Origen:** EstadoGeneralController —
seguridad/estado/controller/EstadoGeneralController.java
 - **Verificación:** EstadoGeneralServiceTest (2 pruebas), EstadoGeneralControllerTest (1 prueba).
-

3.3 Módulo deportivo

Actualizado 2026-07-30. RF-16 (entrenadores) pasó de  Modelado a  Implementado con la reestructuración de paquetes. RF-17 a RF-21 siguen con su **esquema de datos migrado y versionado** (V3__dominio_deportivo.sql , V4__evaluaciones.sql) pero **sin API REST propia todavía**. RF-22 (representantes) y el módulo de equipos no tienen ni esquema: son paquetes Java vacíos (ver nota al final de esta sección).

RF-16 — Gestión de entrenadores El sistema deberá permitir registrar entrenadores asociados a una persona y a una cuenta de usuario, con especialidad, años de experiencia y certificación, garantizando que una misma persona o cuenta no pueda registrarse dos veces como entrenador.

- **Prioridad:** Alta · **Estado:**  Implementado (cambió de Modelado) · **MoSCoW:** Must
- **Origen:** EntrenadorController (/api/entrenadores , 5 endpoints) —
deportivo/entrenador/controller/EntrenadorController.java
- **Esquema:** deportivo.entrenadores , con UNIQUE sobre id_persona e id_usuario (el vínculo con Usuario es nuevo respecto a la v1.0 de este documento).
- **Verificación:** EntrenadorServiceTest (7 pruebas: paginación con mapeo de persona/usuario, persona duplicada, usuario duplicado, alta válida, especialidad inexistente, baja lógica),
EntrenadorControllerTest (5 pruebas: 200/201/204/404/422).

Actualizado 2026-08-12. especialidad pasó de texto libre a un catálogo (deportivo.especialidades , FK id_especialidad , nullable) — ver EspecialidadController / EspecialidadService y EspecialidadServiceTest . El formulario de alta de entrenador en el frontend pasó de un input de texto a un <select> poblado desde GET /api/especialidades/activas .

RF-17 — Horarios recurrentes de entrenamiento  Implementado (2026-08-13) El sistema deberá permitir definir horarios semanales recurrentes por categoría y entrenador, y deberá impedir que la hora de fin sea anterior o igual a la hora de inicio.

- **Prioridad:** Alta · **MoSCoW:** Must — de él se generan las sesiones (RF-18).

POST/GET /api/horarios , DELETE /api/horarios/{id} (baja lógica), todos hasRole('ENTRENADOR') y acotados al propio entrenador autenticado (404 si el horario no es suyo). Esquema: deportivo.horarios_entrenamiento , con CHECK (hora_fin > hora_inicio) y CHECK (dia_semana BETWEEN 1 AND 7) . De aquí se generan solas las sesiones del día que corresponde (ver RF-18): antes, cada sesión — fuera una recurrente o una extra— se creaba a mano.

RF-18 — Sesiones de entrenamiento  Implementado (2026-08-13) El sistema deberá registrar cada sesión de entrenamiento con su fecha, categoría, entrenador responsable y estado, admitiendo únicamente los estados PROGRAMADA, EN_CURSO, FINALIZADA y CANCELADA.

- **Prioridad:** Alta · **MoSCoW:** Must — de ella dependen asistencia (RF-19) e historial (RF-35).

POST /api/sesiones (jornada extra, fuera del horario fijo), GET /api/sesiones/hoy y /mias . Estos dos últimos generan primero, de forma idempotente, la sesión de hoy de cada horario fijo activo que caiga en el día (HorarioService.generarSesionesDeHoy()) antes de listar — así ni el entrenador ni recepción dependen de que

alguien cree la sesión a mano. Esquema: `deportivo.sesiones_entrenamiento` , con restricción `CHECK` sobre `estado` y FK opcional `id_horario` hacia el horario que la originó (null si es una jornada extra).

RF-19 — Registro de asistencia  Implementado (dividido en RF-19a/RF-19b — ver nota)

Corrección (2026-09-07). El enunciado original exigía "el marcaje por RFID o manual" como si fueran una sola capacidad *Must*, cuando en realidad una vía está implementada y la otra no — un *Must* parcialmente cumplido no deja ver, sin leer la prosa, qué parte falta. Se divide en dos requisitos con estado independiente, siguiendo la misma disciplina que ya se aplicó en otras entradas de este documento.

RF-19a — Registro de asistencia por QR o manual  Implementado El sistema deberá registrar la asistencia de cada estudiante a cada sesión mediante código QR (marcado por el propio estudiante) o lista manual (marcada por el entrenador), con estado *PRESENTE*, *TARDE*, *AUSENTE* o *JUSTIFICADO*, y deberá impedir que se registre más de una asistencia del mismo estudiante en la misma sesión.

- **Prioridad:** Alta · **MoSCoW:** Must — precondition de notificaciones (RF-22) e historial (RF-35).

RF-19b — Registro de asistencia por RFID  Planificado El sistema deberá admitir el marcaje de asistencia mediante lector RFID como vía adicional a RF-19a.

- **Prioridad:** Baja · **MoSCoW:** Could — la escuela no dispone hoy de lector físico; sin ese hardware no hay forma de verificar la capacidad aunque se programe. El `CHECK` de `metodo` en el esquema ya admite el valor `'RFID'` (ver más abajo), así que activarla no exige migración, solo el lector y el endpoint.

Esquema (común a ambos): `deportivo.asistencias` , con `UNIQUE (id_sesion, id_estudiante)` y `CHECK` sobre `metodo` y `estado` .

Dos vías de marcaje, y la distinción entre ambas se conserva en el dato:

Vía	Quién marca	metodo	hora_entrada
QR rotativo	el propio estudiante	QR	hora real de llegada, medida
Lista manual	el entrenador	MANUAL	vacía

La lista manual **no escribe una hora de llegada**. El entrenador afirma que el estudiante estuvo, no a qué hora entró; si pasa lista al terminar el entrenamiento, un `LocalTime.now()` guardaría la hora en que tecleó y quedaría escrito como si el chico hubiera llegado dos horas tarde a una sesión a la que llegó puntual. Así la columna significa algo preciso: si hay hora, la midió el QR; si no la hay, es palabra del entrenador.

RFID es RF-19b, arriba: sin lector físico, sin implementar.

Endpoints: `GET /api/asistencias/sesion/{id}` (nómina completa de la categoría, no solo quienes ya marcaron) y `PUT /api/asistencias/sesion/{id}` (upsert idempotente por estudiante). Una sesión con fecha posterior a hoy se puede consultar pero no editar: nadie pudo asistir todavía.

RF-20 — Evaluación diaria del desempeño  Implementado (corregido 2026-09-07 — ver nota) El sistema deberá permitir al entrenador evaluar a cada estudiante por criterios configurables (técnica, condición física, táctica y actitud), registrando la posición jugada ese día, y deberá impedir puntajes negativos y evaluaciones duplicadas del mismo estudiante y criterio dentro de una misma evaluación.

- **Prioridad:** Media · **MoSCoW:** Should
- **Origen:** `EvaluacionDiariaController` (`GET /api/evaluaciones/sesion/{idSesion}` , `PUT /api/evaluaciones/sesion/{idSesion}/jugadores` , `POST`

```
/api/evaluaciones/sesion/{idSesion}/finalizar ) —  
deportivo/evaluacion/controller/EvaluacionDiariaController.java
```

- **Verificación:** EvaluacionDiariaServiceTest , EvaluacionDiariaControllerTest .

Esquema: deportivo.evaluaciones_diarias , deportivo.criterios_evaluacion ,
deportivo.detalle_evaluacion , con CHECK (puntaje >= 0) y UNIQUE (id_evaluacion, id_estudiante,
id_criterio) .

Corrección (2026-09-07). Esta entrada declaraba "solo esquema, sin endpoint todavía" — ya no es cierto: el controlador con sus 3 rutas existe desde antes de esta revisión. El estado no se había actualizado cuando se implementó. RF-21, en cambio, sigue correctamente en Modelado: existe la vista deportivo.v_promedio_evaluacion pero ningún controlador la expone.

RF-21 — Consulta de promedios de evaluación 🟡 Modelado El sistema deberá calcular el promedio de puntajes por estudiante y evaluación en el motor de base de datos.

- **Prioridad:** Media · **MoSCoW:** Should — depende de RF-20, también solo esquema.

Esquema: vista deportivo.v_promedio_evaluacion .

RF-33 — Agenda de partidos ✅ Implementado (2026-08-25) El sistema deberá permitir al entrenador agendar un partido de una categoría y registrar su resultado después de jugado.

- **Prioridad:** Alta · **MoSCoW:** Must — base del dominio de partidos (RF-34).

deportivo.partidos (GET-POST-PUT-DELETE /api/partidos). Los goles admiten nulo y no tienen valor por defecto: un partido recién agendado no va 0-0, todavía no se jugó, y NULL («sin resultado») y 0 («no metió ninguno») son cosas distintas. Un CHECK exige que estén los dos marcadores o ninguno: un marcador a medias no dice si se ganó. GANADO/EMPATADO/PERDIDO/PENDIENTE se calcula, no se almacena, para que no puedan contradecir al marcador.

Solo se lleva el marcador propio. El sistema es de una academia: «local/visitante» exigiría un catálogo de rivales que nadie va a mantener.

RF-34 — Convocatoria y once del partido ✅ Implementado (2026-08-25) El sistema deberá sugerir el once inicial de un partido a partir del rendimiento acumulado de las semanas previas, y deberá permitir al entrenador modificarlo y guardar la formación con la que efectivamente jugó.

- **Prioridad:** Alta · **MoSCoW:** Must — regla de negocio central del módulo deportivo (tope de once titulares, exclusión por lesión).

deportivo.alineaciones + deportivo.alineacion_jugador (GET-PUT-DELETE
/api/partidos/{id}/alineacion).

La IA no elige a los jugadores. La regla es explícita y reproducible a mano: universo = plantel activo de la categoría; quedan fuera —con el motivo a la vista— el lesionado y quien no pisó un entrenamiento en la ventana; se ordena por promedio de evaluación de las últimas 4 semanas (plantilla.semanas-rendimiento), desempatando por presencias y después por id para que dos llamadas con los mismos datos den lo mismo; y se titulariza al mejor de cada posición nominal, no a los once mejores promedios —eso podía sugerir dos porteros y ningún defensa—. El modelo de lenguaje solo redacta un comentario sobre un once **ya decidido**, y solo cuando se le pide.

La ventana es lo que hace que la sugerencia se alimente semana a semana: el promedio histórico completo premia al que jugó bien hace un año por encima del que viene mejor ahora.

Sugerencia y decisión se guardan por separado: **si el entrenador guardó una alineación se devuelve esa; si no, la sugerida**. La sugerencia no se persiste sola —hacerlo convertiría una recomendación en un hecho histórico sin que nadie lo decidiera—.

Hasta V21 la alineación colgaba de la sesión de entrenamiento. V22 la movió al partido: decidir con quién se sale a jugar no es un hecho del entrenamiento, y atarla a la sesión obligaba a que solo pudieran alinearse los que fueron a **ese** entrenamiento.

RF-35 — Historial de asistencia de una sesión  Implementado (2026-08-25) *El sistema deberá mostrar, para una sesión ya ocurrida, quiénes asistieron y quiénes no.*

- **Prioridad:** Media · **MoSCoW:** Should — reporte sobre datos que ya existen por RF-19.

`GET /api/sesiones/{id}/historial` . Se parte del **plantel** de la categoría y no de las filas de asistencia: si nadie pasó lista, la tabla está vacía y una consulta que solo lea de ahí diría «no había nadie convocado», que es distinto de «no se registró la asistencia de nadie». Por eso existe el estado `SIN_REGISTRO` , separado de `AUSENTE` .

RF-22 — Notificación a representantes  Implementado (2026-08-09) *El sistema deberá notificar al representante legal cuando su representado marque asistencia o registre una lesión.*

- **Prioridad:** Media · **MoSCoW:** Should — depende de RF-19/RF-31; hoy solo en-app, no correo/SMS/push (sección Trabajo futuro del informe).

El rol REPRESENTANTE, el vínculo con sus representados y la tabla de consentimientos que este requisito exige como precondition (hallazgo H-04 de `docs/etica/ETHICS.md`) existen desde 2026-08-03. La notificación en sí es **en-app** (tabla `academico.notificaciones` , `NotificacionService`): al marcar asistencia (`AsistenciaService.marcarPorQr`) o registrar una lesión (`LesionService.registrar`) se crea una fila por cada representante con vínculo activo, visible en `GET /api/representante/notificaciones` y marcabale como leída. No es correo/SMS/push — este proyecto no tiene infraestructura de envío externo, y agregarla requeriría credenciales que nadie tiene todavía; una notificación en-app satisface el requisito sin esa dependencia.

Actualización 2026-08-12 (asignación de representados). *Hasta ahora el vínculo representante↔estudiante solo existía en el backend: no había forma en la interfaz de ver ni asignar quién representa a un estudiante. La ficha de estudiante de `/personas` ahora lista los representantes vinculados y permite agregar y quitar, indicando la relación del vínculo y cuál es el `contacto_principal` — que es justamente a quien apunta esta notificación. Ver `docs/superpowers/specs/2026-08-12-coherencia-rol-y-vinculo-representante-design.md` .*

Corrección (2026-09-07). *Esta nota decía que el paquete `academico.representante` estaba vacío (0 bytes) — ya no es cierto, y no lo era desde antes de esta revisión: quedó desactualizada cuando el paquete se llenó. Hoy (renombrado a `academico.guardian` en el rename al inglés de esta entrega) tiene DTOs, repositorios, servicios y entidades con contenido real, y tres controladores con rutas propias: `GuardianController` (`/api/representantes` , 9 rutas: CRUD, reactivación, vincular/desvincular estudiante), `GuardianReportController` (`/api/representante` , 7 rutas: informe del representado, comentario, notificaciones) y `ConsentController` (`/api/consentimientos` , 3 rutas: alta, revocación, consulta — ver A2 más abajo). Ver también RF-24/RF-25 para la gestión de representantes como recurso propio.*

El módulo `deportivo.equipo` mencionado en la versión anterior de esta nota ya no existe ni siquiera como paquete vacío — se eliminó del código. Sigue **Planificado** (fila "Equipo" de la matriz de trazabilidad): sin esquema ni endpoint, solo declarado.

Actualización 2026-08-12. *Al reconciliar la base de Supabase para el módulo Inventario se descubrió que sí existía, creado a mano y fuera de control de versiones, un esquema de tablas para equipos, partidos y ejercicios (`deportivo.equipos` , `deportivo.partidos` , `deportivo.estadistica_partidos` , `deportivo.ejercicios` , `deportivo.entrenamiento_ejercicios`), todas vacías. Se versionó en*

V16__equipos_partidos_ejercicios.sql para que exista igual en cualquier entorno — es la mitad de base de datos de la limpieza de 2026-07-30 que en su momento solo tocó el código. Sigue sin EquipoController ni API REST: el esquema existe, la funcionalidad no. Estado: 🟡 solo esquema, no implementado.

RF-31 — Registro y alta de lesiones ✅ Implementado (2026-08-14) El sistema deberá permitir al entrenador registrar una lesión de un estudiante (descripción y fecha estimada de retorno opcional), impidiendo una segunda lesión activa simultánea del mismo estudiante, y deberá permitir darla de alta cuando el estudiante se recupera.

- **Prioridad:** Alta · **MoSCoW:** Must — condiciona la exclusión de lesionados en RF-34 y es dato de salud sensible (ver ETHICS.md).

Esquema: deportivo.lesiones (LesionController , LesionService); el backend ya existía de una revisión anterior, pero sin frontend que lo consumiera. Se agregaron los botones "Marcar lesión" / "Dar de alta" a la pantalla de Evaluación diaria del entrenador, y se endureció LesionController.registrar : el idEntrenador de una cuenta ENTRENADOR ya no sale del cuerpo de la petición (que un entrenador podía manipular para registrar una lesión "a nombre de" otro), sino que se resuelve del token autenticado, mismo criterio que SesionEntrenamientoController .

RF-32 — Autoconsulta del estudiante sobre su equipo y desempeño ✅ Implementado (2026-08-14) El sistema deberá permitir a un estudiante autenticado consultar su propia categoría, su posición nominal, el entrenador de su próxima sesión programada, sus compañeros de equipo (solo nombre y posición, sin datos de contacto ni promedios — son menores de edad) y sus propias estadísticas de evaluación (promedio histórico por criterio, porcentaje de asistencia de los últimos 30 días e historial de lesiones propio).

- **Prioridad:** Media · **MoSCoW:** Should — transparencia hacia el estudiante, no bloquea ningún otro requisito.

MiEquipoController (GET /api/estudiante/mi-equipo , GET /api/estudiante/mi-informe), solo rol ESTUDIANTE. Las estadísticas reutilizan tal cual la lógica que InformeService ya usaba para el informe que el representante ve de un representado (construirInforme , extraído como método común); el equipo es un endpoint nuevo (MiEquipoService) que cruza academico.estudiantes , deportivo.categorias , deportivo.posiciones y la sesión futura más próxima de deportivo.sesiones_entrenamiento para resolver el entrenador asignado.

3.4 Módulo de inventario (nuevo en esta revisión)

Cierra el schema inventario que ADR-003 había reservado como diseño a futuro. Stock por cantidad agregada (no serializado por unidad individual): cada artículo tiene un stock_actual que los movimientos y las asignaciones ajustan, nunca por debajo de cero.

RF-27 — Gestión del catálogo de artículos de inventario El sistema deberá permitir crear, listar, consultar, actualizar y dar de baja artículos de inventario (uniformes, balones, implementos u otro), cada uno con un stock mínimo configurable para alertas de reposición.

- **Prioridad:** Alta (bloquea RF-28/RF-29) · **Estado:** ✅ Implementado · **MoSCoW:** Must
 - **Origen:** ArtículoController (/api/inventario/articulos , 6 endpoints) — inventario/articulo/controller/ArticuloController.java
 - **Esquema:** inventario.articulos , con CHECK sobre tipo y CHECK (stock_actual >= 0) .
 - **Verificación:** ArtículoServiceTest (6 pruebas: alta con stock en cero, edición sin tocar el stock, baja lógica, paginación, stock bajo).
-

RF-28 — Registro de movimientos de stock El sistema deberá registrar entradas, salidas y ajustes de stock por artículo, y deberá impedir cualquier salida que deje el stock por debajo de cero.

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
- **Origen:** `MovimientoStockController` (`/api/inventario/movimientos` , 3 endpoints) — `inventario/movimiento/controller/MovimientoStockController.java`
- **Esquema:** `inventario.movimientos_stock` , con `CHECK` (`cantidad > 0`) y `CHECK` sobre `tipo_movimiento` .
- **Verificación:** `MovimientoStockServiceTest` (4 pruebas: entrada, ajuste y salida ajustan el stock correctamente; salida que dejaría el stock negativo se rechaza sin persistir nada).

RF-29 — Asignación y devolución de artículos *El sistema deberá permitir asignar artículos a un estudiante o a un entrenador (nunca a ambos en la misma asignación), descontando el stock disponible, y deberá permitir marcar la devolución como DEVUELTO (repone el stock) o PERDIDO (no lo repone).*

- **Prioridad:** Alta · **Estado:**  Implementado · **MoSCoW:** Must
- **Origen:** `AsignacionController` (`/api/inventario/asignaciones` , 5 endpoints) — `inventario/asignacion/controller/AsignacionController.java`
- **Esquema:** `inventario.asignaciones` , con `CONSTRAINT` `chk_asignacion_destinatario` (exactamente un destinatario según `tipo_destinatario`) y `CHECK` sobre `estado` .
- **Verificación:** `AsignacionServiceTest` (9 pruebas: asignación a estudiante y a entrenador, stock insuficiente, destinatario faltante, devolución que repone stock, pérdida que no lo repone, doble resolución, transición inválida a ASIGNADO, asignación inexistente).

RF-30 — Reporte de artículos con stock bajo *El sistema deberá reportar, como cálculo agregado en el motor de base de datos, el total de artículos activos cuyo stock actual esté en o por debajo de su stock mínimo.*

- **Prioridad:** Media · **Estado:**  Implementado · **MoSCoW:** Should
- **Origen:** `GET /api/inventario/articulos/stock-bajo` — combina el listado (JPA) con el conteo agregado del procedimiento almacenado `inventario.sp_reporte_stock_bajo` (ver `docs/basedatos/CATALOGO-SP.md`).
- **Verificación:** `ArticuloServiceTest.stockBajo_combina_listado_y_total_del_procedimiento` .

4. Requisitos no funcionales

Clasificados según las características de calidad de ISO/IEC 25010:2011. Los valores medidos provienen de la evidencia real versionada en `docs/mediciones/` , no de estimaciones.

4.1 Eficiencia de desempeño

RNF-01 — Tiempo de respuesta *El sistema deberá responder a las consultas paginadas de estudiantes con un percentil 95 inferior a 200 ms con caché caliente e inferior a 500 ms con caché fría, bajo una carga de 50 usuarios virtuales concurrentes.*

- **Verificación:** 3 corridas independientes de k6 (50 VUs, 30 s).
- **Resultado medido:** p95 promedio **14,18 ms** (IC 95 % $\pm$ 3,20), media 6,48 ms, throughput 404,57 RPS, **0 % de errores**. Cumple con amplio margen. Evidencia: `docs/mediciones/perf/REPORT.md` y los tres `k6-run*.json` .

RNF-02 — Caché de consultas frecuentes *El sistema deberá cachear las consultas de listado de estudiantes con un tiempo de vida de 60 segundos, y la caché no deberá corromper la deserialización de tipos temporales.*

- **Origen:** `RedisCacheConfig.java` ; `CACHE_TTL_SECONDS=60` .
- **Nota:** se corrigió un defecto real por el cual la caché fallaba desde el segundo request (Jackson no leía el `@class` raíz) y otro por falta de soporte de `java.time.Instant` .

4.2 Seguridad

RNF-03 — Almacenamiento de contraseñas El sistema no deberá almacenar contraseñas en texto plano ni de forma reversible; deberá utilizar BCrypt con factor de coste 12. Verificación: `db/seed.sql` y `SecurityConfig.java`.

RNF-04 — Transporte cifrado El sistema deberá ofrecer acceso mediante HTTPS con TLS 1.2 o superior. Medido: TLS 1.3 vía nginx en `:8443`. Evidencia: `docs/mediciones/sec/a02-tls.txt` (OWASP A02).

RNF-05 — Cabeceras de seguridad El sistema deberá enviar en todas sus respuestas las cabeceras `Content-Security-Policy`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` y, sobre HTTPS, `Strict-Transport-Security`. Evidencia: `docs/mediciones/sec/a05-cabeceras.txt` (OWASP A05).

RNF-06 — Control de acceso por rol El sistema deberá denegar toda petición a recursos protegidos que no presente una sesión válida (`401`) o cuyo rol no esté autorizado para la operación (`403`), y dicha verificación deberá aplicarse del lado del servidor con independencia de lo que muestre la interfaz. Evidencia: `docs/mediciones/sec/a01-acceso-roto.txt` (OWASP A01).

RNF-07 — Ausencia de SQL dinámico El sistema no deberá construir sentencias SQL mediante concatenación de cadenas; toda consulta deberá usar parámetros vinculados o procedimientos almacenados con parámetros nombrados. Verificación: `make audit` incluye auditoría de SQL dinámico. Evidencia: `docs/mediciones/sec/a03-inyeccion.txt` (OWASP A03).

RNF-08 — Registro de auditoría de autenticación El sistema deberá registrar cada intento de autenticación, exitoso o fallido, incluyendo marca de tiempo, dirección IP de origen e identificador del sujeto, sin registrar nunca la contraseña. Evidencia: `docs/mediciones/sec/a09-logging.txt` (OWASP A09).

4.3 Fiabilidad y mantenibilidad

RNF-09 — Cobertura de pruebas El sistema deberá mantener una cobertura de líneas y de ramas (branches) igual o superior al 70 %, verificada automáticamente en la construcción.

Corrección (2026-09-07). El enunciado y la cifra de abajo citaban 60 % de instrucciones — ese nunca fue el valor configurado en `pom.xml` (que exige 70 % en `LINE` y en `BRANCH`, sin excepciones de paquete) y la cifra estaba fechada 2026-07-30, mucho antes del estado actual del código. Cifra vigente, regenerada el 2026-09-07 tras el rename de identificadores de esta entrega (`./mvnw clean verify`): **84,63 % de líneas (2638/3117) y 71,24 % de branches (664/932), 550 pruebas en 74 clases, 200 clases analizadas — CUMPLE el 70 % en ambas métricas.** Desglose por subdominio en `docs/informe/main.tex` (Tabla `tab:cobertura-por-paquete`, 25 filas) y dato crudo en `docs/mediciones/jacoco/jacoco.csv`. La bitácora original de esta jornada (72,5 % el 2026-07-30) se conserva abajo sin alterar, como registro histórico de cómo se llegó hasta acá — no como cifra vigente.

- Medido el 2026-07-30 con construcción limpia (`./mvnw clean test`): **72,5 % (2507 instrucciones cubiertas de 3457) — cifra histórica, no vigente (ver corrección arriba).**
- 102 pruebas en 17 clases, **todas pasan** (0 fallos, 0 errores).
- Por qué "construcción limpia" aparece explícito aquí: la primera medición de esta jornada se hizo con `./mvnw test` sobre un `target/` que aún conservaba `.class` de antes de la reestructuración de paquetes. El reporte archivado llegó a listar paquetes que ya no existen en el código fuente (`org.uteq.backend.auth.*`, `org.uteq.backend.estudiante.*`). La cifra publicada ahora proviene de `clean test`, y el reporte de `docs/mediciones/jacoco/` contiene solo los 27 paquetes reales.
- **Historial de esta cifra en la misma jornada**, para que quede trazable: primero se detectó una regresión real a 39,8 % (los 5 recursos nuevos de la reestructuración — Categoría, Entrenador, Usuario, Persona, EstadoGeneral — no tenían ninguna prueba propia); se agregaron 57 pruebas nuevas (`CategoriaServiceTest`, `CategoriaControllerTest`, `EntrenadorServiceTest`,

EntrenadorControllerTest , UsuarioServiceTest , UsuarioControllerTest ,
PersonaServiceTest , PersonaControllerTest , EstadoGeneralServiceTest ,
EstadoGeneralControllerTest) y la cobertura subió a 72,5 %.

- Evidencia: docs/mediciones/jacoco/ (reporte regenerado con la ejecución que incluye las 101 pruebas).

RNF-10 — Tipificación de errores El sistema deberá responder los errores en formato `ProblemDetail` (RFC 9457), con `type` , `title` , `status` , `detail` e `instance` , y no deberá exponer trazas de pila ni detalles internos de implementación. Origen: `GlobalExceptionHandler.java` , `ProblemDetailsAuthHandlers.java` .

RNF-11 — Versionado del esquema de datos Todo cambio en el esquema de base de datos deberá aplicarse mediante una migración Flyway versionada e incremental; no deberá modificarse el esquema de forma manual ni automática por el ORM en tiempo de arranque. Origen: V1 a V6 ; `ddl-auto: validate` .

4.4 Portabilidad

RNF-12 — Reproducibilidad en un solo comando El sistema deberá levantarse completo (base de datos, caché, backend y frontend) desde una clonación limpia del repositorio mediante un único comando, en menos de dos minutos y sin configuración manual adicional más allá de copiar el archivo de variables de entorno de ejemplo. Origen: `make up` . Verificado mediante clonación real independiente en carpeta separada, no solo reiniciando el volumen local.

RNF-13 — Fijación de dependencias de infraestructura Las imágenes de contenedor de base de datos y caché deberán fijarse por digest SHA-256 y no por etiqueta móvil, para garantizar que dos construcciones del mismo commit usen exactamente los mismos binarios. Origen: `docker-compose.yml` (digests reales aplicados por `scripts/pin-digests.sh`).

5. Trazabilidad

La correspondencia entre cada requisito, su implementación, su prueba automatizada y su evidencia empírica se mantiene en [docs/trazabilidad/matriz.csv](#) .

El seguimiento de las observaciones emitidas por el docente en las entregas previas se mantiene en `docs/observaciones/` .

6. Control de cambios del documento

Versión	Fecha	Entrega	Cambios principales
1.0	2026-05-xx	Entrega 1A	Versión inicial.
1.1	2026-07-15	Entrega 1B / Tercera	Resuelve OBS-01 y OBS-12; se añaden módulos de catálogos, inventario y dominio deportivo.
1.2	2026-08-24	Entrega Final (v1.0.0)	Reestructuración de paquetes academico/deportivo/seguridad; RF-35 e historial de asistencia; cierre de trazabilidad (matriz de 47 filas).
1.3	2026-09-04	Entrega Final (v1.0.0)	Campo MoSCoW explícito en los 36 RF (11 no tenían prioridad formal); matriz de trazabilidad ampliada a 50 filas.

7. Aprobación

Este documento constituye la especificación de requisitos acordada para la Entrega Final del proyecto SGED, cerrada en la etiqueta **v1.0.0** del repositorio.

Rol	Nombre	Firma	Fecha
Autor (equipo)	Arcalle Grefa Darwin Orlando		2026-09-04
Autor (equipo)	Pallo Pinto Alejandro Daniel		2026-09-04
Autor (equipo)	Velez Lopez Ricardo Elias		2026-09-04
Docente evaluador	Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.	_____	_____